

Accor Redemption Platform

Architecture & Design Document

1. Executive Summary

This document describes the architecture design for the Accor Redemption Platform.

The solution follows cloud-native DevOps practices using AWS, Terraform, Kubernetes, and observability tooling.

The platform focuses on scalability, reliability, automation, and reducing operational toil.

Infrastructure is managed through Infrastructure as Code and application workloads are deployed using Kubernetes.

2. Architecture Overview

The platform architecture consists of:

- AWS infrastructure provisioned using Terraform
- Amazon EKS Kubernetes cluster
- Managed Node Group for workload execution
- Kubernetes application deployment
- Traefik ingress routing
- Prometheus, Grafana, and AlertManager monitoring stack

High-level flow:

Users

|

AWS Cloud

|

VPC

|

EKS Cluster

|

Node Group

|

Kubernetes Workloads

|

Redemption Service

3. Architectural Decisions

Terraform:

Terraform was selected to provide repeatable, version-controlled infrastructure provisioning. It enables automation of AWS resources and reduces manual configuration.

Amazon EKS:

EKS provides a managed Kubernetes control plane and integrates with AWS services, reducing Kubernetes operational overhead.

Kubernetes:

Kubernetes was selected for container orchestration because it provides self-healing, rolling deployments, service discovery, and horizontal scaling capabilities.

Observability:

Prometheus and Grafana provide metrics collection and visualization, while AlertManager enables proactive operational alerting.

4. Kubernetes Design

Application Deployment:

The redemption service runs as a Kubernetes Deployment with three replicas.

Benefits:

- High availability
- Pod failure recovery
- Rolling updates

Service:

A ClusterIP service provides stable internal communication between workloads.

Ingress:

Traefik Ingress Controller manages external traffic routing.

Configuration:

ConfigMaps store application configuration while Secrets manage sensitive information.

5. Reliability and Scaling

Horizontal Pod Autoscaler:

Configuration:

- Minimum replicas: 3
- Maximum replicas: 10
- CPU target: 70 percent

The HPA automatically scales workloads based on resource usage.

PodDisruptionBudget:

Minimum available pods: 2

This protects application availability during maintenance operations.

6. Monitoring and Alerting

Monitoring stack:

- Prometheus
- Grafana
- AlertManager

Dashboards monitor:

- Pod availability
- CPU usage
- Memory usage
- Container restarts
- HPA status

Custom alerts:

- RedemptionPodDown
- RedemptionHighCPU
- RedemptionContainerRestart

7. Day 2 Operations Strategy

The platform minimizes operational toil through:

Infrastructure Automation:

- Terraform modules

- Version-controlled infrastructure
- Repeatable deployments

Kubernetes Automation:

- Self-healing workloads
- Automated scaling
- Rolling deployments

Observability:

- Metrics dashboards
- Alert rules
- Proactive incident detection

Future improvements:

- CI/CD automation
- GitOps deployment
- Centralized logging
- Automated security scanning

8. Team Delegation Plan

Senior Engineer:

Responsibilities:

- Overall architecture design
- AWS infrastructure design
- Terraform modules
- EKS architecture
- Security review
- Code reviews

Junior Engineer 1:

Responsibilities:

- Kubernetes manifests
- Deployment configuration

- Services
- Ingress
- ConfigMaps
- Secrets

Junior Engineer 2:

Responsibilities:

- Monitoring setup
- Grafana dashboards
- Prometheus alerts
- Documentation
- Testing

9. Architecture Trade-offs

EKS:

Benefit:

Managed Kubernetes control plane.

Trade-off:

Higher cost compared to self-managed Kubernetes.

Terraform:

Benefit:

Infrastructure automation and repeatability.

Trade-off:

Requires Terraform expertise.

Kubernetes HPA:

Benefit:

Automatic scaling.

Trade-off:

Requires proper metrics configuration.

Prometheus:

Benefit:

Powerful open-source monitoring.

Trade-off:

Requires storage and retention planning.

10. Conclusion

The Accor Redemption Platform architecture provides a scalable, reliable, and operationally efficient foundation. The design combines AWS infrastructure automation, Kubernetes orchestration, and observability practices suitable for production workloads.